설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

인덱싱과 슬라이싱

posco

설비 고장을 예측하는 AI 데이터 분석

문자열 다루기

문자열의 이해와 생성 Part 1

문자열 만들기

자료형과 형변환

인덱싱과 슬라이싱 Part 2

인덱싱

슬라이싱

연산·검색과 정리 메서드 Part 3

정리 메서드

f-string과 텍스트 정리 Part 4

f-string

인덱싱과 슬라이싱

posco

글자에도 위치 번호가 있다

문자열의 글자 하나하나에는 **자리 번호**가 매겨져 있음

극장 좌석처럼 각 글자에 번호가 붙어 있음

이 번호로 원하는 글자만 콕 집어 꺼내기 가능

인덱싱이란

인덱싱 - 위치 번호로 글자 하나를 꺼내기 (영어 indexing)

변수 뒤 대괄호 안에 번호 - word[2]

책 뒤 색인처럼 위치를 가리키는 개념

주의 - 인덱싱은 글자 한 개만 꺼냄

인덱스는 0부터 시작한다

가장 중요한 규칙 - 첫 글자의 번호는 0

'PYTHON'에서 P는 0번, Y는 1번, N은 5번

글자가 6개라도 번호는 0부터 5까지

하나만 기억한다면 바로 이것

인덱스 번호표

글자	P	Y	T	H	O	N
양수	0	1	2	3	4	5
음수	-6	-5	-4	-3	-2	-1

양수 인덱스로 글자 꺼내기

변수 뒤 대괄호에 **번호**를 넣어 그 자리 글자 꺼내기

word[0]은 첫 글자, word[2]는 세 번째 글자

번호와 실제 글자를 하나씩 짝지어 확인

양수 인덱싱 - 예시

앞에서부터 **0번**부터 번호를 매겨 글자를 꺼냄

word의 0번 글자는 P, 2번 글자는 T

설비 코드의 0번 글자로 종류 확인 가능

마지막 글자의 인덱스

마지막 글자의 번호는 **글자 수 - 1**

'PYTHON'은 6글자, 마지막 N은 5번

글자 수와 같은 번호(6번)는 없는 자리 - 오류

실습 1. 양수 인덱스로 글자 꺼내기

목표	양수 인덱스로 원하는 자리의 글자 꺼내기 (0부터)
단계	① 'SIGNAL' 저장 → ② 0·2·5번 글자를 각각 출력
예상 결과	S / G / L

실습 1. 양수 인덱스로 글자 꺼내기 정답

CODE

```
word = "SIGNAL"
print(word[0])   # S  (첫 글자, 0번)
print(word[2])   # G  (2번)
print(word[5])   # L  (마지막, 5번)
```

음수 인덱스 - 뒤에서부터

음수 인덱스 - 뒤에서부터 세는 번호

맨 뒤 글자는 -1, 그 앞은 -2

앞은 0부터, 뒤는 -1부터

글자 수를 몰라도 마지막 글자를 word[-1]로

실습 2. 음수 인덱스로 뒤에서 꺼내기

목표	음수 인덱스로 뒤에서부터 글자 꺼내기
단계	① 'SIGNAL' 저장 → ② 예상 결과와 동일하게 나오도록 뒤에서 꺼내기
예상 결과	L / A

실습 2. 음수 인덱스로 뒤에서 꺼내기 정답

CODE

```
word = "SIGNAL"
print(word[-1])   # L  (마지막)
print(word[-2])   # A  (끝에서 둘째)
```

인덱스 범위를 벗어나면

Q1. 없는 번호를 쓰면 어떻게 되나요?

A. IndexError 오류 발생

Q2. 언제 자주 생기나요?

A. 0부터 세는 걸 깜빡하거나, 마지막을 글자 수와 같게 쓸 때

Q3. 만나면 어떻게 하나요?

A. 인덱스를 0부터 다시 세어 보기

인덱싱 활용 - 코드 첫 글자

설비 코드의 **첫 글자**로 종류를 빠르게 확인

'P'로 시작하면 펌프, 'M'이면 모터 식으로 구분

code[0]으로 분류 기준 글자 추출

한계 - 한 글자만 꺼냄 (구간은 슬라이싱 필요)

슬라이싱이란

슬라이싱 - 여러 글자를 구간으로 잘라내기

대괄호 안에 시작:끝 형태로 범위 지정

전화번호 가운데 네 자리, 코드 앞 세 글자처럼

인덱싱은 글자 하나, 슬라이싱은 구간 하나

슬라이싱 문법 [start:end]

대괄호 안에 **콜론**을 찍어 시작:끝 지정

word[1:4]는 1·2·3번 자리 (시작 포함, 끝 제외)

중요 규칙 - 시작은 포함, 끝은 제외

0부터 시작 규칙도 그대로 적용

슬라이싱 문법 - 코드 예시

시작 포함, **끝 제외** 규칙

CODE

```
word = 'PYTHON'
print(word[1:4])   # YTH  (1,2,3번 / 4 제외)
print(word[0:3])   # PYT
code = 'EQP-001'
print(code[0:3])   # EQP  (앞 세 글자)
print(code[4:7])   # 001  (뒤 숫자)
```

end는 포함되지 않는다

끝 번호는 **포함되지 않음** (그 직전까지만)

word[1:4]는 4번을 빼고 1·2·3번만

공식 - 가져올 마지막 글자 번호

+ 1을 end에

가장 자주 하는 실수이자 가장 중요한 규칙

실습 3. 구간 자르기

목표	[start:end] 로 원하는 구간 자르기 (end는 미포함)
단계	① 'SIGNAL' 저장 → ② 예상 결과와 동일하게 나오도록 구간 자르기
예상 결과	SIG / GNA

실습 3. 구간 자르기 정답

CODE

```
word = "SIGNAL"
print(word[0:3])   # SIG  (0,1,2번 — 3은 미포함)
print(word[2:5])   # GNA
```

start 생략 - 앞에서부터

시작을 비우면 **0번부터**라는 뜻

word[:3]은 0·1·2번 - word[0:3]과 완전히 동일

앞부분을 가져올 때 시작 번호를 생략

실습 4. 앞부분 가져오기

목표	start 생략 [:n] 으로 앞부분 가져오기
단계	① 'temp_sensor' 저장 → ② 4글자만 추출하기
예상 결과	temp

실습 4. 앞부분 가져오기 정답

CODE

```
word = "temp_sensor"
print(word[:4])    # temp  (앞 4글자, start 생략=처음부터)
```

end 생략 - 끝까지

끝을 비우면 **맨 끝까지** 가져옴

word[3:]은 3번부터 마지막 글자까지

뒷부분을 가져올 때 끝 번호를 생략

실습 5. 뒤쪽 전체 가져오기

목표	end 생략 [n:] 으로 뒤쪽 전체 가져오기
단계	① 'temp_sensor' 저장 → ② 5번부터 끝까지 추출
예상 결과	sensor

실습 5. 뒤쪽 전체 가져오기 정답

CODE

```
word = "temp_sensor"
print(word[5:])    # sensor  (5번부터 끝까지, end 생략)
```

앞뒤 생략 - 전체 가져오기

시작도 끝도 비우면 **전체**를 가져옴

word[:]은 처음부터 끝까지 전부

앞부터 + 끝까지 두 규칙을 합친 당연한 결과

음수 인덱스로 슬라이싱

음수 번호로 **뒤에서부터** 구간 자르기

word[-3:]은 뒤에서 세 글자

word[:-1]은 마지막 한 글자를 뺀 나머지

음수에도 끝 제외 규칙이 그대로 적용

실습 6. 뒤 n글자 가져오기

목표	음수 인덱스 슬라이싱 [-n:] 으로 뒤 n글자 가져오기
단계	① 'sensor_01' 저장 → ② 뒤 2글자 출력
예상 결과	01

실습 6. 뒤 n글자 가져오기 정답

CODE

```
word = "sensor_01"
print(word[-2:])   # 01  (뒤 2글자)
```

step - 건너뛰며 자르기

콜론을 **하나 더** 찍어 몇 칸씩 건너뛸지 지정

word[::2]는 0·2·4번 (두 칸씩)

세 번째 칸이 step, 기본값은 1

step - 코드 예시

세 번째 칸이 **step** (건너뛸 간격)

CODE

```
word = 'PYTHON'
print(word[::2])    # PTO  (0,2,4번)
print(word[1::2])   # YHN  (1번부터 두 칸씩)
print(word[::1])    # PYTHON  (간격 1 = 전체)
```

실습 7. 건너뛰며 가져오기

목표 step [::2] 으로 한 칸씩 건너뛰며 가져오기

단계 ① 'SIGNAL' 저장 → ② 두 칸씩 건너뛴 글자만 출력

예상 결과 SGA

실습 7. 건너뛰며 가져오기 정답

CODE

```
word = "SIGNAL"
print(word[::2])   # SGA  (0,2,4번 — 2칸씩)
```

step 음수 - 문자열 뒤집기

step에 **음수**를 넣으면 뒤에서부터 가져옴

word[::-1]은 문자열 전체를 뒤집기

'PYTHON'이 'NOHTYP'로

뒤집기 공식처럼 통째로 기억

실습 8. 문자열 뒤집기

목표	step 음수로 문자열을 거꾸로 뒤집기
단계	① 'SIGNAL' 저장 → ② 저장한 문자열을 뒤집어 출력
예상 결과	LANGIS

실습 8. 문자열 뒤집기 정답

CODE

```
word = "SIGNAL"
print(word[::-1])   # LANGIS  (뒤집기)
```

슬라이싱은 범위 벗어나도 오류 없음

Q1. 슬라이싱은 범위를 넘으면 오류가 나나요?

A. 오류 없음 - 가능한 만큼만 가져옴

Q2. 인덱싱과 어떻게 다른가요?

A. 인덱싱은 엄격(없는 번호=오류), 슬라이싱은 너그러움

인덱싱 vs 슬라이싱 정리

콜론이 있으면 슬라이싱, 없으면 인덱싱

인덱싱은 글자 하나, 슬라이싱은 구간

이 한 가지 기준만 잡으면 절대 안 헷갈림

인덱싱 vs 슬라이싱 비교표

기준	인덱싱	슬라이싱
대괄호 안	번호 하나	시작:끝 (콜론)
꺼내는 것	글자 한 개	구간 여러 글자
예시	word[0]은 P	word[0:3]은 PYT
범위 초과	오류 발생	오류 없음
0부터·음수	동일 적용	동일 적용

활용 - 날짜·번호·코드 구간 추출

자리 규칙이 정해진 데이터를 부분으로 쪼개기

날짜 '20250115'를 연[:4]·월[4:6]·일[6:]로

전화번호·코드의 정해진 자리를 슬라이싱으로 추출

감사합니다.

posco